

MiniChess AI – Design, Search, and a Neural-Network Evaluation Study

Mini Project 2

Student ID: 114000110

June 19, 2026

Abstract

This report documents a strong AI for the 6×5 “MiniChess” variant and is written to be self-contained: it explains not only *what* the engine does but *why* each choice is correct, so every design decision can be defended without referring to the source. The engine combines a classical alpha-beta / Principal Variation Search (PVS) with a full modern search stack (transposition table, killer/counter/history move ordering, iterative deepening, aspiration windows, null-move pruning, late-move reductions, futility pruning, quiescence search) and a *hybrid evaluation* that blends a hand-crafted heuristic with a small *NNUE* (Efficiently Updatable Neural Network) trained from scratch on self-play data. Beyond the working engine, the report presents an empirical study: porting Stockfish’s NNUE architecture ideas (SFNNv1–v14), int16 quantization, texel tuning, and a search-allocation optimization. The central, repeatedly-confirmed finding is that **on a tiny board, search depth dominates evaluation sophistication**, and that **training-loss improvements do not imply playing-strength improvements**. The final engine beats every provided baseline (weak/strong minimax, alpha-beta, PVS) with both colours and sweeps the visible boss 8/8. Section 12 is a TA-question crib sheet.

Contents

1	Overview and architecture	1
2	Game rules, board representation, and move generation	2
2.1	Rules (and the one rule that shapes everything)	2
2.2	Board representation and move encoding	2
2.3	Move generation: bitboards on 30 bits	2
2.4	State transition and the node pool	3
3	Position keys: Zobrist hashing and repetition	3
4	State value function (evaluation)	4
4.1	Hand-crafted evaluation (shipped, <code>custom_evaluate</code>)	4
4.2	NNUE evaluation and the blend	4
5	Tree search	4
5.1	From minimax to negamax	4
5.2	Alpha-beta pruning	5
5.3	Principal Variation Search (PVS)	5
5.4	Quiescence search	6

5.5	Transposition table (TT)	6
5.6	Move ordering	6
5.7	Selectivity: pruning, reductions, extensions	7
5.8	Iterative deepening, aspiration windows, time management	7
6	NNUE: a neural evaluation trained from scratch	7
6.1	Feature encoding (side-to-move relative)	7
6.2	Network and why it is small	8
6.3	Training pipeline	8
6.4	Inference: sparse, per-leaf, float	8
6.5	Blend tuning	9
7	Architecture study: porting Stockfish NNUE (SFNNv1–v14)	9
7.1	What changed across SFNN versions	9
7.2	Dual-perspective accumulator (SFNNv1 core)	9
7.3	int16 quantization (auto-vectorized)	10
7.4	Conclusion of the study	10
8	Search-speed optimization: a ply-indexed node pool	10
9	Negative results (also useful at the demo)	10
10	Results and methodology	10
11	Complexity and memory summary	11
12	Anticipated TA questions and answers	11
13	Key points to remember for the demo	13

1 Overview and architecture

The program speaks **UBGI** (a UCI-superset protocol) over stdin/stdout. The code is split into game-agnostic and game-specific layers:

- `src/ubgi/ubgi.cpp` – protocol loop and the *iterative deepening* driver (calls the search at depth 1, 2, 3, ..., honours `go movetime`, emits `info/bestmove`).
- `src/games/minichess/state.cpp` – board representation, move generation (naive + bit-board), Zobrist hashing, terminal detection, the reference hand-crafted evaluation.
- `src/policy/submission.cpp` – the engine we developed: PVS search, all pruning/ordering heuristics, the tuned hand-crafted evaluation, and the NNUE blend. (This is the policy that competes; algorithm name `submission`.)
- `src/policy/minimax.cpp` – a clean reference PVS+quiescence used to explain the core algorithm and to generate early training games.
- `src/policy/nnue.hpp`, `nnue_weights.hpp` – the neural evaluator and its embedded weights (plain C++ arrays).
- `tools/nnue_gen.cpp` – a C++ self-play data generator that labels positions with the strong `submission` search; `tools/train_nnue.py` – a PyTorch (GPU) trainer that consumes the generated data and emits the embedded weights. (Training is offline only; the engine itself uses no external ML dependencies.)

The big picture. Each turn the driver runs iterative deepening; at every node the search applies alpha-beta/PVS with aggressive but provably-safe pruning; at the leaves a quiescence search resolves captures and then calls the hybrid evaluation (hand-crafted + NNUE). Move generation, hashing, and the evaluation are all expressed from the *side-to-move*’s perspective so the search is a single symmetric negamax.

2 Game rules, board representation, and move generation

2.1 Rules (and the one rule that shapes everything)

MiniChess is chess on a 6-row $\times$ 5-column board with a **king-capture** win model: *a side wins the instant it can capture the enemy king on its turn*. There is no castling and no en passant; a pawn moves one square forward (no double step) and promotes *only to a queen* on the last rank. Draws occur by 3-fold repetition and by a step cap; if the cap is hit the winner is decided by summed piece value.

The king-capture model is the single most important simplification, and it drives several design choices (Section 12):

- **There is no “illegal move”.** You are *allowed* to move into a square where your king can be captured next turn – you simply lose then. Therefore every move produced by the generator is legal, and **the engine can never lose by outputting an illegal move.**
- **No checkmate/stalemate detection is needed.** A terminal “win” is detected purely as “a generated move lands on the enemy king.” Losing is handled implicitly: if we leave our king capturable, the opponent’s reply node is a win *for them*, which negamax turns into a loss for us.

2.2 Board representation and move encoding

The board is two planes, one per player:

```
char board[2][6][5]; // board[player][row][col]; values:
// 0 empty, 1 Pawn, 2 Rook, 3 Knight, 4 Bishop, 5 Queen, 6 King
```

A Move is `pair<Point,Point> = (from, to)` with `Point = pair<size_t,size_t> = (row, col)`. Promotion is implicit: a pawn reaching row 0 (White) or row 5 (Black) is rewritten to a queen inside `next_state`. Row 0 is the top; White starts on rows 4–5 and advances toward row 0.

2.3 Move generation: bitboards on 30 bits

$6 \times 5 = 30$ squares fit in a `uint32_t`, square $(r, c) \rightarrow \text{bit } r \cdot 5 + c$. At startup we precompute leaper attack masks (knight, king, pawn-push, pawn-capture). Generation builds occupancy masks, then bit-scans own pieces with `_builtin_ctz`; leapers index their mask, sliders (rook/bishop/queen) walk direction rays until blocked. The naive array version (`get_legal_actions_naive`) is kept for reference; the engine uses the bitboard path (`USE_BITBOARD`).

Listing 1: Sliding-piece generation + early king-capture win (bitboard)

```
int d_start = (piece==4)?4:0, d_end = (piece==2)?4:8; // R:0-3 B:4-7 Q:0-7
for(int d=d_start; d<d_end; d++){
    int cr=r+bb_dr[d], cc=c+bb_dc[d];
    while(in_bounds(cr,cc)){
        uint32_t to_bit = 1u << BB_SQ(cr,cc);
```

```

    if(self_occ & to_bit) break; // own piece blocks
    if((oppn_occ & to_bit) && oppn_pt[to]==6){// captures enemy KING:
        game_state = WIN; // -> we win right now
        legal_actions.push_back(Move({r,c},{cr,cc}));
        return; // short-circuit: nothing else matters
    }
    targets |= to_bit;
    if(oppn_occ & to_bit) break; // capture stops the ray
    cr += bb_dr[d]; cc += bb_dc[d];
}
}

```

The `game_state=WIN; return;` pattern is the terminal detector: the moment any piece can take the enemy king, generation stops and flags a win, so the search treats the node as a won leaf.

2.4 State transition and the node pool

`next_state(move)` copies the board, applies the move (with promotion), incrementally updates the Zobrist hash, flips the side to move, and returns a new `State`. For speed the search instead uses `apply_into(dst,move)` / `null_into(dst)`, which write the child position *in place* into a caller-owned pool slot, avoiding a heap allocation per node (Section 8).

3 Position keys: Zobrist hashing and repetition

Each (piece, colour, square) and the side-to-move own a random 64-bit key (xorshift-seeded once). A position’s hash is the XOR of the keys of all occupied squares, plus the side key if Black is to move. The hash is updated *incrementally* on each move – XOR out the moving piece at its source, XOR out any captured piece, XOR in the (possibly promoted) piece at the destination, and toggle the side key:

Listing 2: Incremental Zobrist update in `next_state`

```

uint64_t h = this->hash();
h ^= zobrist_side; // side to move flips
h ^= zobrist_piece[p][orig_piece][from.r][from.c]; // remove from source
if(captured) h ^= zobrist_piece[opp][captured][to.r][to.c]; // remove victim
h ^= zobrist_piece[p][moved][to.r][to.c]; // add at destination (moved may be promoted Q)

```

This makes the hash $O(1)$ per move instead of $O(30)$. The hash keys the transposition table (Section 5.5) and the repetition map.

Repetition / draws. `GameHistory` is a hash $\rightarrow$ count map; the search pushes the current hash before recursing and pops it after. `check_repetition` returns a draw (0) when a position has occurred **3 times**, implementing the 3-fold rule and, crucially, stopping the search from looping forever in a repetition.

4 State value function (evaluation)

The evaluation returns an integer in centipawn-like units *from the side-to-move’s perspective* (so the search is a pure negamax). The shipped engine (`submission.cpp`) uses a **blend** of a tuned

hand-crafted term and the NNUE. (The reference `minimax.cpp/state.cpp` use a simpler hand-crafted eval – material+PST+tropism+mobility – which is useful to contrast at the demo but is not what competes.)

4.1 Hand-crafted evaluation (shipped, custom_evaluate)

For each side we sum, then take self – opponent:

$$S_{\text{hand}} = (\text{material} + \text{PST} + \text{tropism} + \text{pawns})_{\text{self}} - (\dots)_{\text{opp}} + w_m \cdot |\text{legal moves}|, \quad (1)$$

with these components:

- **Material** $\{P, R, N, B, Q, K\} = \{100, 500, 320, 330, 900, 10000\}$. The king’s 10000 is a sentinel; real king safety comes from search, not this number.
- **Piece-square tables (PST)**: a per-piece 6×5 table, read from the side-to-move’s orientation (Black’s row is mirrored, `pst_r = player? r: 5-r`). Knights/bishops/queens are pushed toward the centre; the king is pushed back (negative centre, positive home squares).
- **King tropism**: own non-king pieces get $w_t[\text{type}] \cdot \max(0, 4 - \text{Chebyshev dist to enemy king})$ with weights $\{P, R, N, B, Q\} = \{2, 5, 8, 5, 10\}$ – rewards crowding the enemy king, which directly ends the game.
- **Pawn structure**: doubled pawns –15 each extra on a file; isolated pawns (no friendly pawn on an adjacent file) –10.
- **Passed pawns**: $\max(10, 50 - 10 \cdot \text{dist-to-promotion})$ – large because promotion to a queen is decisive on a small board.
- **Mobility**: $w_m = 5$ times the side-to-move’s legal-move count (re-using the already-generated `legal_actions`, so it is free).

Terminal positions short-circuit before any of this: a win returns P_{max} , a draw returns 0 (and the search adjusts wins to $P_{\text{max}} - \text{ply}$, below).

4.2 NNUE evaluation and the blend

The NNUE (Section 6) produces a score from the same side-to-move perspective; it is blended with the hand-crafted score:

$$S = \frac{b \cdot S_{\text{nnue}} + (100 - b) \cdot S_{\text{hand}}}{100}, \quad b = \text{NNUE_BLEND} = 45. \quad (2)$$

NNUE is on by default and self-contained; `-DNO_NNUE` disables it and `-DNNUE_BLEND=n` overrides the mix. **Why a blend, and why 45%?** Empirically (Section 6.5) pure NNUE is *weaker* than the hand-crafted eval on this game, but a partial admixture is meaningfully stronger than either component alone: the net captures outcome-correlated patterns the heuristic misses, while the heuristic supplies tactical robustness and is cheap.

5 Tree search

5.1 From minimax to negamax

Minimax assumes the opponent minimises our score. Because MiniChess strictly alternates turns and the evaluation is symmetric (side-to-move relative), we use the *negamax* identity $\text{value}(n) = \max_m - \text{value}(\text{child})$: each side maximises its own perspective, and a child’s score is negated when read by the parent.

Listing 3: Negamax with alpha-beta (concept)

```
int negamax(node, depth, alpha, beta):
    if terminal(node): return WIN? P_MAX-ply : (DRAW? 0 : ...) // mate-distance aware
    if depth == 0: return quiescence(node, alpha, beta)
    for each move m (in ordering priority):
        score = -negamax(child(m), depth-1, -beta, -alpha)
        if score >= beta: return beta // fail-high: opponent avoids this line
        if score > alpha: alpha = score // new best
    return alpha
```

Mate scoring. A win returns $P_{\max} - \text{ply}$ (not just $P_{\max}$). Subtracting the ply makes *shorter* mates score higher, so the engine prefers to win quickly and to delay losing – and it lets mate scores be compared meaningfully across the tree. TT storage adds the ply back on store and removes it on probe so a mate found via a transposition keeps the correct distance.

King-capture and the “same player” flag. The generic search supports games where the mover does not alternate (e.g. Connect6) via `same_player_as_parent()`; MiniChess always alternates, so the child is always negated. Because a capturable enemy king makes `get_legal_actions` flag WIN, the negamax terminal check fully captures both winning (our move takes their king) and losing (their reply takes ours, negated) without any special-casing.

5.2 Alpha-beta pruning

α is the best score the side to move is already assured of; β is the best the opponent is assured of. When $\alpha \geq \beta$ the remaining moves cannot affect the result (the opponent would never enter this line), so the branch is pruned. Alpha-beta returns the *exact same* value as minimax but, with good ordering, visits $O(b^{d/2})$ instead of $O(b^d)$ nodes – effectively doubling reachable depth.

5.3 Principal Variation Search (PVS)

PVS assumes the first (best-ordered) move is best. It searches that move with the full window $[\alpha, \beta]$ and every later move with a *null window* $[\alpha, \alpha+1]$ – a cheap boolean test “is this move worse than the best so far?”. If a null-window search surprises us by failing high, that move is re-searched with the full window. When ordering is good, almost all moves are refuted by the cheap test, so PVS is strictly faster than plain alpha-beta with no loss of correctness.

Listing 4: PVS core (from submission.cpp, simplified)

```
if(move_index == 0){
    score = -search(child, depth-1, -beta, -alpha); // full window on PV move
}else{
    score = -search(child, depth-1, -alpha-1, -alpha); // null-window scout
    if(score > alpha && score < beta) // scout failed high
        score = -search(child, depth-1, -beta, -alpha); // re-search full window
}
```

5.4 Quiescence search

At depth 0 the engine does not evaluate immediately; it runs a *quiescence* search over “noisy” moves (captures and promotions) until the position is quiet, avoiding the *horizon effect* (evaluating in the

middle of a capture sequence). It uses (i) a **stand-pat** cutoff – the static eval is a lower bound because the side to move can usually decline to capture; (ii) MVV-LVA ordering; and (iii) **delta pruning** – skip a capture if $\text{stand-pat} + V_{\text{victim}} + \text{margin} < \alpha$ (it cannot raise α even in the best case).

Listing 5: Quiescence stand-pat and delta pruning

```
int stand_pat = custom_evaluate(state);
if(stand_pat >= beta) return beta; // already good enough
if(stand_pat + (QUEEN+200) < alpha) return alpha; // whole-node delta cutoff
if(alpha < stand_pat) alpha = stand_pat;
// ... then try only captures/promotions, MVV-LVA ordered ...
```

5.5 Transposition table (TT)

A $2^{22} \approx 4 \cdot 10^6$ -entry hash table caches search results keyed by the Zobrist hash. Each `TTEntry` (≈ 10 bytes; ≈ 40 MB total) stores a 32-bit key check (to detect index collisions), score, depth, a bound flag (**exact/lower/upper**), a 16-bit packed best move (`CompactMove`, 4 bits per coordinate), and a generation byte for ageing. The TT (i) returns a usable score when the position recurs at $\geq$ the stored depth *and* the bound is compatible with the current window, and (ii) feeds the *TT move* to move ordering – the single biggest ordering win.

Listing 6: Bound-aware TT probe (why a cached score is usable)

```
if(entry.depth >= depth){
    if(entry.flag == TT_EXACT) return score; // exact value
    if(entry.flag == TT_ALPHA && score <= alpha) return alpha; // upper bound <= alpha
    if(entry.flag == TT_BETA && score >= beta) return beta; // lower bound >= beta
}
```

An *exact* entry is the true value; a *lower* bound (fail-high) is only trusted when it already exceeds β ; an *upper* bound (fail-low) only when it is $\leq \alpha$. This is exactly the information alpha-beta proved when the entry was stored, so reusing it changes nothing about correctness.

5.6 Move ordering

Good ordering is what makes alpha-beta/PVS effective. Moves are tried in this priority (higher score first):

1. **TT move** – the best move previously found here ($\approx 10^7$).
2. **Captures by MVV-LVA** (Most Valuable Victim, Least Valuable Attacker: $16 V_{\text{victim}} - V_{\text{attacker}}$).
3. **Promotions**.
4. **Killer moves**: two quiet moves per ply that recently caused a cutoff at the same ply.
5. **Counter move**: the quiet move that previously refuted the opponent’s last move (indexed by that move’s from/to).
6. **History heuristic**: quiet moves scored by how often they caused cutoffs, accumulated per (side, from, to) as $+d^2$, halved on overflow.

Killers/counters/history are pure *ordering* hints; they never prune, so a bad hint costs a little speed but cannot change the result.

5.7 Selectivity: pruning, reductions, extensions

On top of exact alpha-beta the engine searches *selectively*. Every technique below is disabled in the situations where it would be unsound:

- **Null-move pruning (NMP)**: give the opponent a free move; if the position is still $\geq \beta$ at reduced depth ($R = \min(d-1, 3+[d \geq 7])$, clamped ≥ 2), prune. Disabled in check and at PV nodes; unproven mate scores from a null search are clamped to β (never returned as a real mate). Rationale: passing is almost always worse than the best real move, so if even a free move for the opponent can't drag us below β , no real move will.
- **Late-move reductions (LMR)**: late, quiet, non-killer moves are searched at reduced depth (reduction 1–4, growing with move index and depth, -1 at PV nodes). If a reduced search beats α it is re-searched at full depth, so a reduction can only *save time*, never drop a move that was actually good.
- **Reverse futility / static null move**: at low depth, if $\text{static} - 120 d \geq \beta$, return early (the position is so good a full search is unnecessary). Off in check / PV.
- **Futility pruning**: at low depth, skip quiet moves when $\text{static} + 150 d \leq \alpha$ (they cannot plausibly raise α).
- **Late-move pruning (LMP)**: at very low depth, skip quiet moves beyond a move-count threshold.
- **Mate-distance pruning**: tighten the window with known mate bounds so the engine prefers faster mates and never wastes effort proving a slower one.
- **Check extension**: search one ply *deeper* when in check, so forcing lines are not cut off by the depth limit.
- **Internal iterative deepening (IID)**: at a PV node with no TT move, do a shallow search first to obtain a good first move for ordering.

5.8 Iterative deepening, aspiration windows, time management

The driver searches depth 1, 2, ..., reusing the previous depth's TT and PV for ordering (so a deeper search is usually *faster* per node than starting cold). From the previous score it opens a narrow *aspiration window* (± 50); on a fail-high/low it widens the window exponentially and re-searches. Time is checked every 2048 nodes; the search aborts shortly before the per-move budget using a margin of $\min(300, \text{movetime}/25)$ ms, and a “half-time” rule avoids starting a depth that almost certainly cannot finish. **The competition limit is a strict 2 s per move**; the engine adapts to whatever *movetime* the runner passes and always has a legal move ready (Section 12, “losing on time”).

6 NNUE: a neural evaluation trained from scratch

No pre-trained weights or runtime ML libraries are used; the engine embeds the weights as C++ arrays. Training is offline (PyTorch on GPU) and feeds data generated by a C++ self-play tool.

6.1 Feature encoding (side-to-move relative)

The board is canonicalised to the side to move so one set of weights serves both colours. White keeps orientation; **Black is rotated** 180° ($r \rightarrow 5-r$, $c \rightarrow 4-c$), which is exact because Black's start

array is White’s rotated 180°. Each occupied square contributes one binary feature:

$$\text{feature} = \underbrace{\text{base}}_{\substack{0 \text{ own, } 180 \text{ enemy}}} + (\text{piece} - 1) \cdot 30 + \text{sq}, \quad \text{sq} = \begin{cases} r \cdot 5 + c & \text{White} \\ (5-r) \cdot 5 + (4-c) & \text{Black} \end{cases} \quad (3)$$

giving $2 \times 6 \times 30 = 360$ features (own/enemy $\times$ 6 piece types $\times$ 30 squares). *Worked example:* a White rook on ($r=5, c=0$) is own piece, type 2, $\text{sq} = 25$, so $\text{feature} = (2 - 1) \cdot 30 + 25 = 55$. The encoding is implemented *identically* in `nnue_gen.cpp` and `nnue.hpp` – a mismatch would silently corrupt the evaluation, so this is the first thing to check if the net ever “goes crazy.”

6.2 Network and why it is small

The shipped network is a fully-connected $360 \rightarrow 32 \rightarrow 1$ with ReLU and a tanh output (scaled by 600 to centipawns). The width is a deliberate *speed* decision: because the accumulator is rebuilt at every search leaf (below), wider nets cost search depth. Measured at 1.5s from the start position:

Hidden width H	32	64	128	256
Depth reached	15	13	11	11

$H=128$ fits the data better (lower validation MSE) yet, in head-to-head play, was *no stronger* and sometimes worse – the ≈ 4 lost plies cancel the better eval. $H=32$ is the strength/speed optimum, so we kept it and improved the *weights* instead.

6.3 Training pipeline

1. **Self-play data** (`tools/nnue_gen.cpp`): $\approx 30\text{k}$ games are played by the strong `submission` search (depth-7 iterative deepening; random openings of 1–8 plies plus 10% random moves for diversity), yielding $\approx 926\text{k}$ labelled positions. Six processes generate shards in parallel; each non-terminal position stores its active features, side to move, the teacher search score, and (after the game) the result.
2. **Labels:** each position carries its eventual game result (WDL, $\pm 1/0$ from its side-to-move’s perspective) *and* the teacher search score. The target blends them, $t = \lambda \text{WDL} + (1 - \lambda) \tanh(\text{score}/400)$ with $\lambda=0.7$ – WDL keeps the net honest about outcomes, the teacher score sharpens positional nuance.
3. **Training** (`tools/train_nnue.py`, PyTorch on GPU): Adam, mini-batch MSE against the tanh output, cosine-decayed learning rate, 80 epochs, 4% validation hold-out (best-val checkpoint kept; final validation MSE ≈ 0.36).
4. **Embedding:** weights are written to `nnue_weights.hpp` as plain C++ arrays (with W_1 laid out feature-major so the inner loop is contiguous), so the engine is self-contained and compiles with the standard command – **no third-party library is linked into the engine.**

6.4 Inference: sparse, per-leaf, float

The board has ≤ 20 pieces, so the accumulator is rebuilt *sparingly from scratch* at every leaf – summing only the ≤ 20 active feature rows of W_1 – rather than maintained incrementally. This trades a little speed for zero make/unmake accumulator bugs (a classic NNUE pitfall), and on this board the eval is still cheap. The output is

$$\text{cp} = \text{round}\left(\tanh\left(b_2 + \sum_h W_2[h] \cdot \text{relu}(b_1[h] + \sum_{f \text{ active}} W_1[f][h])\right) \cdot 600\right). \quad (4)$$

Listing 7: NNUE forward pass (nnue.hpp, simplified)

```
float acc[H];
for(int h=0; h<H; h++) acc[h] = B1[h]; // bias
for(each occupied square) { // <= 20 iterations
    int f = feature_index(piece, r, c, stm); // own/enemy split, 180-flip for black
    const float* w = &W1[f*H];
    for(int h=0; h<H; h++) acc[h] += w[h]; // add this feature's row
}
float pre = B2;
for(int h=0; h<H; h++) pre += W2[h] * relu(acc[h]);
return lround(tanh(pre) * 600); // centipawns, stm perspective
```

6.5 Blend tuning

We measure blend ratios with *diverse openings* (Section 10). Pure NNUE is weaker than the hand-crafted eval here; a partial admixture is best. For the retrained net we swept $b \in \{30, 45, 60\}$ against a fixed reference engine: $b=45$ scored best (and never worse), so it is the shipped default. Against the previous $b=30$ net (same $360 \rightarrow 32 \rightarrow 1$ architecture, $\approx 98k$ training positions) the retrained $b=45$ engine scores **60%** over 30 games with mild openings, at *identical* search speed. It sweeps the visible boss **8/8** both from the fixed start and with diverse openings at 2s.

7 Architecture study: porting Stockfish NNUE (SFNNv1–v14)

We investigated whether Stockfish’s NNUE architecture evolution transfers to MiniChess. This is a likely demo discussion point and a deliberate set of *negative* results.

7.1 What changed across SFNN versions

Version	Key architectural idea	Transfers to MiniChess?
SFNNv1	HalfKP features; <i>two-perspective</i> accumulator; clipped ReLU; deeper head; int quantization	dual-persp / clipped-ReLU: yes; HalfKP: too many features
SFNNv4	HalfKAv2 (king included as a piece)	partial (we include the king)
SFNNv5	HalfKAv2.hm (horizontal mirroring)	no – board is not L/R symmetric
SFNNv6	L1 width doubled; <i>piece-count output buckets</i>	buckets: cheap, plausible
v7–v14	L1 scaling $1024 \rightarrow 3072$, more data	scaling only

7.2 Dual-perspective accumulator (SFNNv1 core)

We implemented the defining NNUE idea: encode the position from *both* the side-to-move’s and the opponent’s viewpoint through a *shared* feature transformer, concatenate the two accumulators, and pass through a deeper head ($360 \rightarrow 64[\times 2] \rightarrow 32 \rightarrow 1$) with clipped ReLU. Trained on pure WDL it reached a much lower MSE (0.237 vs 0.56) – yet it **played worse**: the two accumulators plus the bigger head cut NPS $\sim 3\times$, costing ~ 1.5 –2 plies of depth. The lower loss did not translate into strength.

7.3 int16 quantization (auto-vectorized)

To recover the lost speed we quantized to int16 (SFNNv1’s speed trick): weights scaled by $Q_A=512$ (clipped-ReLU unit) and $Q_W=256$ (layer weights), with *int32* accumulation laid out contiguously so the compiler auto-vectorizes under `-march=native` (no hand-written SIMD; float-vs-int parity ≈ 2.5 cp). This recovered much of the speed but the dual-perspective net was *still* below the simple single-perspective net. Quantizing the single-perspective net gave *no* speedup (its float eval is already cheap and int16 widening cancels the lane-count gain).

7.4 Conclusion of the study

Stockfish’s NNUE architecture is sound but does not transfer to MiniChess. On full chess, NNUE’s eval cost is amortised by enormous search trees, the hand-crafted alternative is far weaker, and int8 SIMD + incremental updates keep NNUE cheap. On a 6×5 board none of that holds: a cheap eval plus more search depth wins, so the structural sophistication (and even the quantization speed trick) is a net loss. The shipped engine therefore uses the *single-perspective float* net.

8 Search-speed optimization: a ply-indexed node pool

Profiling pointed at per-node memory churn: the search did `new State+delete` (and a fresh move-list allocation) for every node. We added `State::apply_into / null_into`, which write the child position *in place* into a caller-owned `State` taken from a ply-indexed pool (`g_node_pool[ply]`), reusing each slot’s move-list buffer. Depth-first search guarantees `pool[child_ply]` is free to overwrite once a subtree returns, and a parent never reads a child slot while recursing – so the search tree is *provably identical* (verified: same node counts and scores) while two heap allocations per node disappear. This is the one optimization (of three attempted) that shipped, because a speedup must be a provable non-regression before it is trusted.

9 Negative results (also useful at the demo)

- **Wider NNUE** ($H=64/128/256$): lower validation loss, no strength gain – the lost search depth cancels it. Kept $H=32$.
- **Texel-tuning the material values** to fit WDL improved the static loss by only 0.4% and *lost* head-to-head: static-eval loss is a poor proxy for strength, and the material values also drive move ordering and delta pruning, so reweighting hurt search efficiency.
- **Opening book**: with a strict *per-move* 2s limit (no game clock) a book cannot bank time and the engine already plays the fixed opening strongly, so a book would only echo the engine’s own move. Not shipped (it would be a no-op).
- **Pure search-score teacher for NNUE**: reached a very low MSE but regressed in play – the net merely learned to imitate the existing eval rather than predict outcomes. (We kept a *blend* of WDL and teacher score instead.)

10 Results and methodology

Measuring strength correctly. Both engines are deterministic, so games from the fixed start position are effectively one game repeated – a “20-game match” from the start cannot resolve

small differences. We therefore use *random openings* (`--opening-ply N --seed S`) in the match runner; all comparative numbers use diverse openings, and matches are run sequentially (time-based results need an unloaded CPU).

Opponent	Result (final engine)	Notes
Weak MiniMax (baseline)	win, both colours	advances baseline
Strong MiniMax (baseline)	win, both colours	advances baseline
Alpha-Beta (baseline)	win, both colours	weaker than the PVS boss
PVS (visible boss)	8/8	2s, fixed start
PVS boss, diverse openings	8/8	2s, random openings (harder)
Previous NNUE ($b=30$)	60%	30 games, mild openings, equal speed

Table 1: Final engine ($H=32$ retrained, blend 45, node pool) vs. opponents.

Constraints respected. Standard library only (NNUE weights are embedded C++ arrays; PyTorch is used *offline* for training and is never linked into the engine); no manual SIMD/inline-asm (only compiler auto-vectorization via `-march=native`); the engine’s policy code is single-threaded; TT ≈ 40 MB, well under the 4 GB limit.

11 Complexity and memory summary

Component	Cost	Notes
Move generation	$O(30)$ bit-scan + ray walks	bitboard, leaper masks precomputed
<code>next_state</code> /hash	$O(1)$ amortised	incremental Zobrist; in-place into pool
Search (ordered AB/PVS)	$\approx O(b^{d/2})$	$b \approx 10\text{--}20$ here
Transposition table	2^{22} entries, ≈ 40 MB	32-bit key check + ageing
NNUE inference	≤ 20 feature rows $\times 32 + 32$	rebuilt per leaf, float
History/killers/counters	$O(1)$ updates	ordering only, never prune

12 Anticipated TA questions and answers

Q. Why negamax instead of separate min/max code?

The evaluation is side-to-move relative and turns strictly alternate, so max for one side is min for the other. Negamax expresses both with one routine by negating the child’s score; less code, fewer bugs.

Q. Does alpha-beta change the move minimax would pick?

No. Alpha-beta only prunes branches that provably cannot affect the value at the root (a side would never choose to enter them). It returns the identical minimax value; it is purely a speed optimization.

Q. How do you know the engine never plays an illegal move (instant loss)?

The move generator only ever produces legal moves, and in this king-capture variant there is no “moving into check is illegal” rule – every generated move is legal. The engine always returns a move from `legal_actions`, so an illegal move is structurally impossible.

Q. How is winning/losing detected without checkmate logic?

Generation flags WIN the instant a move can capture the enemy king, and the search treats that node as a won leaf ($P_{\max} - \text{ply}$). Losing needs no separate code: if we leave our king capturable, the opponent's reply node is a win for them, which negamax negates into a loss for us.

Q. Why $P_{\max} - \text{ply}$ rather than a flat $P_{\max}$?

So the engine prefers the *fastest* win and the *slowest* loss, and so mate scores remain comparable across depths. The TT adds/removes the ply on store/probe so a mate seen through a transposition keeps the right distance.

Q. Is null-move pruning safe? What about zugzwang?

NMP assumes passing is worse than your best real move; that can fail in zugzwang. We disable NMP in check and at PV nodes, require depth ≥ 3 , and never return an *unproven* mate from a null search (clamp to β). On this material-rich small board zugzwang is rare, and the guards make the rare case safe.

Q. Can LMR/futility/LMP make you miss the best move?

LMR re-searches at full depth whenever a reduced search beats α , so a good move is never lost – only confirmed-bad moves are searched shallowly. Futility/LMP only skip *quiet* moves at *low* depth when the static eval is far below α ; captures, promotions, killers, and in-check nodes are exempt. These are heuristics that trade a tiny tactical risk for much greater depth, which on this board is a strong net positive.

Q. Why is a cached TT score trustworthy?

Each entry records the bound type alpha-beta proved: *exact* values are reused directly; a *lower* bound only when it already exceeds β ; an *upper* bound only when $\leq \alpha$ – exactly the conditions under which the cached search would have cut off anyway. A 32-bit key check guards against index collisions.

Q. What is quiescence search for, and why is stand-pat valid?

It resolves captures past the depth horizon so we don't evaluate mid-trade. Stand-pat (the static eval) is a valid lower bound because the side to move can normally decline to capture, so the true value is at least the static score; if that already $\geq \beta$ we cut.

Q. How do you avoid losing on time?

Iterative deepening always has a completed shallower result, time is checked every 2048 nodes and the search stops with a safety margin before the budget, and the protocol layer emits the best move found so far on **stop**. So a legal move is always ready before the 2s cutoff.

Q. Why blend NNUE with a hand-crafted eval instead of using NNUE alone?

Measured: pure NNUE is weaker here, and the hand-crafted eval alone is weaker than the 45% blend. The net adds outcome-correlated positional knowledge; the heuristic adds tactical robustness and is cheap. The blend beats either alone.

Q. Why such a small net ($360 \rightarrow 32 \rightarrow 1$)? Isn't bigger better?

Bigger nets fit the data better but the per-leaf eval is rebuilt every node, so width costs search depth: $H=128$ lost ≈ 4 plies at 1.5s and was no stronger. Depth dominates eval sophistication on a 6×5 board, so $H=32$ is optimal.

Q. You used PyTorch – isn't that a forbidden third-party library?

PyTorch is used *offline* to train; it produces a header of plain `float` arrays. The competing engine links only the C++ standard library. Training tools are not part of the submitted engine.

Q. Why encode features from the side-to-move’s perspective?

One weight set then serves both colours, halving the parameters and doubling effective data. Black’s start is exactly White’s rotated 180°, so a single rotation makes the mapping exact.

Q. How is the NNUE trained “from scratch”?

Self-play games (C++ generator using our own search) label positions with their game result (and teacher score); a small MLP is trained to predict that. No external datasets or pre-trained weights.

Q. What stops infinite loops / repetitions?

A Zobrist-keyed history map; the third occurrence of a position returns a draw, both implementing the 3-fold rule and terminating cyclic search lines.

Q. Why bitboards on a 5-wide board?

All 30 squares fit in a `uint32_t`; a single `__builtin_ctz` bit-scan plus precomputed leaper masks replaces nested array scans, and slider rays are short. It is both faster and simpler than the naive generator (which we keep for reference).

Q. What is the single most important factor in your engine’s strength?

Search depth, enabled by good move ordering (TT move + MVV-LVA + killers/history) and safe pruning. The eval and NNUE refine the leaves, but reaching one more ply reliably beats a smarter but slower evaluation here.

13 Key points to remember for the demo

1. **Search:** negamax + alpha-beta + PVS + quiescence, plus TT, killer/counter/history ordering, IID, aspiration windows, NMP, LMR, reverse-futility/futility/late-move pruning, mate-distance pruning, and check extensions. Be ready to explain *why* each prunes safely (Section 12).
2. **Evaluation:** material + PST + king tropism + pawn structure + passed pawns + mobility, *blended* with a from-scratch NNUE at 45%.
3. **NNUE:** 360-feature side-to-move-relative encoding (180° flip for Black), $360 \rightarrow 32 \rightarrow 1$; trained on $\approx 926k$ positions from strong-search self-play (C++ generator) with a PyTorch GPU trainer, on a blended WDL + teacher-score target; weights embedded in a header; sparse per-leaf inference.
4. **King-capture model:** no checkmate/legality logic needed; the engine cannot output an illegal move; “win” = a move takes the king.
5. **Headline insight:** *on a small board, depth beats eval sophistication*; SOTA chess NNUE architecture and its quantization trick do *not* transfer (measured, not assumed).
6. **Second insight:** *lower training loss $\neq$ stronger play* – shown three times (wider NNUE, dual-perspective NNUE, texel tuning).
7. **Methodology:** use random openings to measure strength; a speedup (node pool) must be a provable non-regression (identical node counts) before shipping.

Build.

```
g++ --std=c++2a -Wall -Wextra -Wpedantic -g -O3 -march=native -static \
-Isrc/games/minichess -Isrc/state -Isrc \
-o build/minichess-ubgi \
src/games/minichess/state.cpp src/policy/minimax.cpp \
src/policy/random.cpp src/policy/submission.cpp src/ubgi/ubgi.cpp
```

NNUE is on by default and self-contained; `-DNO>NNUE` disables it and `-DNNUE.BLEND=n` changes the mix. Retrain with: `build tools/nnue_gen.cpp`, run several shards (`nnue_gen out.bin games depth seed`), then `python tools/train_nnue.py --hidden 32`.